

Design and Analysis of Computer Algorithms

Mock Midterm

Solution Version

Course: Design and Analysis of Computer Algorithms

Exam: Midterm

Source: Sample Midterm Exam

Year: 2026

Note: Time: 120 minutes Total: 120 pts

1. (20 pts)

In all of the recurrences shown below, it is assumed that $T(n) = d$ for some constant d for $n \leq 2$. State, using the asymptotic notation, the solution to each of the recurrences shown below. Just state the answers and do not need to justify them.

- (1) $T(n) = T(n-1) + \frac{1}{n}$
- (2) $T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\log n}$
- (3) $T(n) = 4T\left(\frac{n}{2}\right) + n^2\sqrt{n}$
- (4) $T(n) = 2T\left(\frac{n}{4}\right) + \sqrt{n}$

Reference Answer

- (1) $T(n) = \Theta(\log n)$
- (2) $T(n) = \Theta(n \log \log n)$
- (3) $T(n) = \Theta\left(n^{\frac{5}{2}}\right)$
- (4) $T(n) = \Theta(\sqrt{n} \log n)$

Remark

Master Theorem. 對 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$, $a \geq 1, b > 1$:

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{if } f(n) = O(n^{\log_b a - \varepsilon}) \text{ for some } \varepsilon > 0 \\ \Theta(n^{\log_b a} \log n) & \text{if } f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)) & \text{if } f(n) = \Omega(n^{\log_b a + \varepsilon}) \text{ for some } \varepsilon > 0 \text{ and regularity holds} \end{cases}$$

Extended Master Theorem (Case 2 推廣). 若 $f(n) = \Theta(n^{\log_b a} \log^k n)$:

$$T(n) = \begin{cases} \Theta(n^{\log_b a} \log^{k+1} n) & \text{if } k > -1 \\ \Theta(n^{\log_b a} \log \log n) & \text{if } k = -1 \\ \Theta(n^{\log_b a}) & \text{if } k < -1 \end{cases}$$

各小題：

(1) 展開得

$$T(n) = \sum_{k=1}^n \frac{1}{k} = H_n = \Theta(\log n).$$

- (2) **Extended Master Theorem**, $k = -1$. $a = 2, b = 2$, 故 $n^{\log_b a} = n$. $f(n) = \frac{n}{\log n} = n \cdot \log^{-1} n$, 符合 $f(n) = \Theta(n^{\log_b a} \cdot \log^k n)$ 且 $k = -1$, 故 $T(n) = \Theta(n \log \log n)$.
- (3) **Master Theorem Case 3**. $a = 4, b = 2$, $n^{\log_b a} = n^2$. $f(n) = n^{\frac{5}{2}} = \Omega(n^{2+\varepsilon})$ (取 $\varepsilon = \frac{1}{2}$). 正規性驗證：

$$a \cdot f\left(\frac{n}{b}\right) = 4 \cdot \left(\frac{n}{2}\right)^{\frac{5}{2}} = \frac{n^{\frac{5}{2}}}{\sqrt{2}} \leq \frac{1}{\sqrt{2}} f(n).$$

故 $T(n) = \Theta(n^{\frac{5}{2}})$.

- (4) **Master Theorem Case 2 (標準)**. $a = 2, b = 4$, $n^{\log_4 2} = n^{\frac{1}{2}} = f(n)$, 兩者相等 ($k = 0$), 故 $T(n) = \Theta(\sqrt{n} \log n)$.

2. (30 pts)

Please consider the following equalities. Mark by True or False each of the following statements. You don't need to prove it.

- (1) $\sum_{i=1}^n \sqrt{i} = O(n^{\frac{3}{2}})$
- (2) $n^n = O(2^n)$
- (3) $\frac{n^2}{\log} n = \Theta(n^2)$
- (4) $33n^3 + 4n^2 = \Omega(n^2)$
- (5) $f(n) = \Theta(g(n)) \wedge (\forall n \in \mathbb{N}^+, f(n) \geq 1 \wedge g(n) \geq 1) \implies \lg f(n) = \Theta(\lg g(n))$
- (6) $\sum_{i=1}^n i \lg i = \Theta(n^2 \lg n)$

Reference Answer

- (1) True.
- (2) False.
- (3) False.
- (4) True.
- (5) True.
- (6) True.

Remark

比值極限判斷漸近關係。設 $f, g: \mathbb{N} \rightarrow \mathbb{R}_{>0}$, 且

$$L = \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$$

存在。則可依 L 的值判斷：

- $0 < L < \infty \implies f(n) = \Theta(g(n))$ 。由極限定義，存在 N 使得 $n > N$ 時

$$\left| \frac{f(n)}{g(n)} - L \right| < \frac{L}{2},$$

因此

$$\frac{L}{2} g(n) < f(n) < \frac{3L}{2} g(n).$$

取 $c_1 = \frac{L}{2}$ 、 $c_2 = \frac{3L}{2}$ 、 $n_0 = N$, 符合 Θ 的定義。

- $L = 0 \Rightarrow f(n) = o(g(n))$ ，從而 $f(n) = O(g(n))$ 。由極限定義，對任意 $\varepsilon > 0$ ，存在 N 使得 $n > N$ 時

$$\frac{f(n)}{g(n)} < \varepsilon.$$

這正是 $f(n) = o(g(n))$ 的定義。由於小 o 蘊含大 O ，且大 O 只要求漸近上界存在、不要求 tight，因此也有 $f(n) = O(g(n))$ 。

- $L = \infty \Rightarrow f(n) = \omega(g(n))$ ，從而 $f(n) = \Omega(g(n))$ 。對任意 $M > 0$ ，存在 N 使得 $n > N$ 時

$$\frac{f(n)}{g(n)} > M.$$

這正是 $f(n) = \omega(g(n))$ 的定義。由於小 ω 蘊含大 Ω ，且大 Ω 只要求漸近下界存在、不要求 tight，因此也有 $f(n) = \Omega(g(n))$ 。

若比值極限不存在 (如 $\frac{f(n)}{g(n)}$ 振盪)，上述規則不適用，需回到 O, Ω, Θ 的常數界定義逐一驗證。

- (1) 用積分估算：

$$\sum_{i=1}^n \sqrt{i} \approx \int_1^n \sqrt{x} dx = \left[\frac{2}{3} x^{\frac{3}{2}} \right]_1^n = \Theta(n^{\frac{3}{2}}).$$

故為 $O(n^{\frac{3}{2}})$ 。

- (2) 用比值的 limit 檢查上界：

$$\lim_{n \rightarrow \infty} \frac{n^n}{2^n} = \lim_{n \rightarrow \infty} \left(\frac{n}{2} \right)^n = \infty.$$

若 $n^n = O(2^n)$ ，依定義必須存在正常數 c 使 $n^n \leq c \cdot 2^n$ 對充分大的 n 成立，也就是比值 $\frac{n^n}{2^n}$ 必須最終被常數上界住。但上述 limit 為 ∞ ，所以不是 $O(2^n)$ 。

- (3) 用比值的 limit 檢查下界：

$$\lim_{n \rightarrow \infty} \frac{\frac{n^2}{\log n}}{n^2} = \lim_{n \rightarrow \infty} \frac{1}{\log n} = 0.$$

因此 $\frac{n^2}{\log n} = o(n^2)$ 。若 $\frac{n^2}{\log n} = \Theta(n^2)$ ，依定義必須存在正常數 c 使 $\frac{n^2}{\log n} \geq c \cdot n^2$ 對充分大的 n 成立，但上述 limit 為 0，所以 $\Omega(n^2)$ 不成立，故不是 $\Theta(n^2)$ 。

- (4) $33n^3 + 4n^2 \geq n^2$ 對所有 $n \geq 1$ 成立，取 $c = 1, n_0 = 1$ 即符合 $\Omega(n^2)$ 定義。(事實上為 $\Omega(n^3)$ ， Ω 只是下界。)

- (5) 由 $f(n) = \Theta(g(n))$ 得 $c_1 g(n) \leq f(n) \leq c_2 g(n)$ ，取對數：

$$\lg c_1 + \lg g(n) \leq \lg f(n) \leq \lg c_2 + \lg g(n).$$

因 $f, g \geq 1$ 保證對數有意義且非負。當 $g(n) \rightarrow \infty$ 時，常數項 $\lg c_1, \lg c_2$ 可被吸收；若 $f, g = \Theta(1)$ ，則 $\lg f, \lg g$ 均為有界正常數， Θ 關係仍成立。

- (6) 積分估算：

$$\sum_{i=1}^n i \lg i \approx \int_1^n x \lg x dx = \Theta(n^2 \lg n).$$

(精確計算： $\left[\frac{x^2 \lg x}{2} - \frac{x^2}{2 \ln 2} \right]_1^n \approx \frac{n^2 \lg n}{2}$ 。)

另一思路是用夾擠：上界方面，對所有 $1 \leq i \leq n$ ，有 $\lg i \leq \lg n$ ，所以

$$\sum_{i=1}^n i \lg i \leq \sum_{i=1}^n i \lg n = \lg n \cdot \frac{n(n+1)}{2} = O(n^2 \lg n).$$

下界方面，只看後半段即可。當 $i \geq \lceil \frac{n}{2} \rceil$ 時， $i \geq \frac{n}{2}$ 且 $\lg i \geq \lg(\frac{n}{2})$ ，因此

$$\sum_{i=1}^n i \lg i \geq \sum_{i=\lceil \frac{n}{2} \rceil}^n i \lg i \geq \sum_{i=\lceil \frac{n}{2} \rceil}^n \frac{n}{2} \lg\left(\frac{n}{2}\right).$$

後半段至少有 $\frac{n}{2}$ 項，所以

$$\sum_{i=\lceil \frac{n}{2} \rceil}^n \frac{n}{2} \lg\left(\frac{n}{2}\right) \geq \frac{n}{2} \cdot \frac{n}{2} \cdot (\lg n - 1) = \Omega(n^2 \lg n).$$

由上下界可得

$$\sum_{i=1}^n i \lg i = \Theta(n^2 \lg n).$$

3. (20 pts)

Suppose A is a problem with a well-known lower bound $\Omega(T(n))$ and B is a problem of which the lower bound is unknown. However, we can find a $O(n)$ -time reduction from A to B . Mark by True or False each of the following statements:

- (1) If B can be solved in $O(n^2)$, then A can be solved in $O(n^2)$.
- (2) If A can be solved in $O(n^2)$, then B can be solved in $O(n^2)$.
- (3) If $T(n) = \log n$, then B has an $\Omega(\log n)$ time lower bound.
- (4) If B has an $\Omega(\log n)$ time lower bound, then $T(n) = \log n$.

Reference Answer

Since A can be reduced to B in $O(n)$ time:

- (1) True.
- (2) False.
- (3) False.
- (4) False.

Remark

Reduction 的核心邏輯：A 可在 $O(n)$ 內 reduce 到 B ，代表「任何能解 B 的演算法，都能在 $O(n + t_{B(n)})$ 內解 A 」。這讓我們可以把 B 的上界傳給 A ，或把 A 的下界傳給 B ，但方向和條件各有限制：

- (1) **True**。 B 可在 $O(n^2)$ 解，則 A 可在 $O(n + n^2) = O(n^2)$ 解。✓
- (2) **False**。 Reduction 方向是 $A \rightarrow B$ ，無法反推 B 的上界。得知 A 容易不代表 B 容易。
- (3) **False**。 下界傳遞的前提是 reduction 的代價小於 A 的下界，才能讓剩餘代價由 B 承擔。此處 reduction 需要 $O(n)$ ，而 A 的下界僅 $\Omega(\log n)$ ， $O(n) \gg \Omega(\log n)$ ，reduction 本身已「覆蓋」整個下界，無法對 B 推出任何非平凡結論。反例：若 B 可在 $O(1)$ 解決， A 仍可在 $O(n)$ 解，不違反 A 的 $\Omega(\log n)$ 下界，故 B 不需要有 $\Omega(\log n)$ 下界。
- (4) **False**。 B 的下界是關於 B 本身的性質，無法反推 A 的下界 $T(n)$ 為何。

4. (15 pts)

Suppose that we would like to construct a Max-heap H using bottom-up fashion statically and all the keys are in a complete binary tree T represented by an array in sequence:

7, 16, 42, 21, 37, 53, 18

(1) (5 pts) Which key is the parent of key 37 in the given complete binary tree T ?

(2) (5 pts) Mark by True or False the following statement:

For key 37 in H , the key of its left child is smaller than the key of its right child.

(3) (5 pts) What is the time complexity of such a construction on n keys?

Reference Answer

- (1) The parent of key 37 in the original complete binary tree T is 16.
- (2) The bottom-up max-heap construction can produce [53, 37, 42, 21, 16, 7, 18]. In this heap, key 37 has left child 21 and right child 16, so the statement $21 < 16$ is false.
- (3) The time complexity of bottom-up heap construction on n keys is $\Theta(n)$.

Remark

- (1) 陣列表示法中，位置 i 的 parent 在位置 $\lfloor \frac{i}{2} \rfloor$ 。37 在位置 5， $\lfloor \frac{5}{2} \rfloor = 2$ ，位置 2 的 key 為 16。
- (2) Bottom-up heapify 從最後一個 internal node (位置 $\lfloor \frac{n}{2} \rfloor = 3$) 往前處理：

步驟	陣列狀態
初始	[7, 16, 42, 21, 37, 53, 18]
Heapify(3): $42 < 53$, swap	[7, 16, 53, 21, 37, 42, 18]
Heapify(2): $16 < 37$, swap	[7, 37, 53, 21, 16, 42, 18]
Heapify(1): $7 < 53$, swap	[53, 37, 7, 21, 16, 42, 18]
繼續 sift-down(3): $7 < 42$, swap	[53, 37, 42, 21, 16, 7, 18]

最終 37 在位置 2，左子 (位置 4) 為 21，右子 (位置 5) 為 16， $21 > 16$ ，故 False。

- (3) 直覺上看似 $O(n \log n)$ ，但精確分析：高度 h 的節點有 $O(\frac{n}{2^h})$ 個，每個 heapify 代價 $O(h)$ ，總代價

$$\sum_{h=0}^{\lfloor \log n \rfloor} O\left(\frac{n}{2^h}\right) \cdot h = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right) = O(n).$$

5. (15 pts)

Suppose the following is a divide-and-conquer algorithm for some problem:

“Make the input of size n into 3 subproblems of sizes $\frac{n}{2}$, $\frac{n}{4}$, $\frac{n}{8}$, respectively with $O(n)$ time; Recursively call on these subproblems; and then combine the results in $O(n)$ time. The recursive call returns when the problems become of size 1 and the time in this case is constant.”

- (a) (5 pts) Let $T(n)$ denote the worst-case running time of this approach on the problem of size n . Please express the running time with the recurrence.
- (b) (5 pts) Use a recursion tree to derive the asymptotic bound of $T(n)$ with Big Theta (Θ) notation.
- (c) (5 pts) Please use the substitution method to verify the asymptotic bound you derive in (b).

(Note: $T(n) = \Theta(n) \iff T(n) = O(n)$ and $T(n) = \Omega(n)$.)

Reference Answer

- (a) The recurrence is

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + T\left(\frac{n}{8}\right) + cn,$$

where $c > 0$ and $T(1) = \Theta(1)$.

- (b) In the recursion tree, the total subproblem size at level 1 is

$$\frac{n}{2} + \frac{n}{4} + \frac{n}{8} = 7\frac{n}{8}.$$

Thus the work at level i is at most $cn\left(\frac{7}{8}\right)^i$, and the total work is

$$cn\left(1 + \frac{7}{8} + \left(\frac{7}{8}\right)^2 + \dots\right).$$

This is a convergent geometric series, so $T(n) = \Theta(n)$.

- (c) First prove $O(n)$. Assume for every smaller $m < n$ that $T(m) \leq Cm$. Then

$$T(n) \leq C\left(\frac{n}{2}\right) + C\left(\frac{n}{4}\right) + C\left(\frac{n}{8}\right) + cn = C\left(7\frac{n}{8}\right) + cn.$$

If $C \geq 8c$, then $C\left(7\frac{n}{8}\right) + cn \leq Cn$, so $T(n) = O(n)$.

For the lower bound,

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + T\left(\frac{n}{8}\right) + cn \geq cn,$$

so $T(n) = \Omega(n)$. Therefore, $T(n) = \Theta(n)$.

Remark**思路引導**

- (a) 注意「 $O(n)$ to split」與「 $O(n)$ to combine」合計仍是 $O(n)$ ，不需要相加兩次；三個子問題大小直接讀題目即可。
- (b) Recursion tree 的重點在於每一層的總工作量，而非個別節點。三個子問題大小分別為 $\frac{n}{2}, \frac{n}{4}, \frac{n}{8}$ ，同一層總和為 $7\frac{n}{8}$ ，形成公比 $\frac{7}{8} < 1$ 的等比級數，故整棵樹的工作量收斂為 $O(n)$ 。注意此遞迴樹不是平衡的（各子樹深度不同），只需估算上界即可得 Θ ，因為 $T(n) \geq cn$ 顯然成立。
- (c) Substitution 要明確寫出：猜測 $T(n) \leq Cn$ ，代入遞迴式，化簡後決定 C 的條件 ($C \geq 8c$)，再補 $\Omega(n)$ 的下界。下界的論證只需一行： $T(n) \geq cn$ 因為題目本身的 cn 項已是下界。

6. (10 pts)

A sequence $\langle a_0, a_1, \dots, a_{n-1} \rangle$ is called unimodal if there exists a t such that $\langle a_t, a_{t+1}, \dots, a_{t+n-1} \rangle$ strictly increases and then strictly decreases, where subscript calculations are performed modulo n .

Thus, if the sequence $\langle a_0, a_1, \dots, a_{n-1} \rangle$ is rotated to the left t positions, it strictly increases to a maximum and then strictly decreases.

Examples of unimodal sequences are:

- $\langle 3, 5, 7, 10, 15, 8, 6, 4 \rangle$
- $\langle 6, 3, 1, 5, 8, 10, 12, 15, 13, 11, 7 \rangle$

Design an efficient algorithm to find the maximum value in a unimodal sequence.

(Hint: Use divide-and-conquer.)

Reference Answer

Use a divide-and-conquer binary search. For a midpoint mid , define

$\text{prev} = (\text{mid} - 1 + n) \bmod n$, $\text{next} = (\text{mid} + 1) \bmod n$.

If $a[\text{mid}]$ is greater than both neighbors, it is the maximum. If $a[\text{mid}] < a[\text{next}]$, the sequence is still increasing locally, so the maximum is in the right half. Otherwise, the maximum is in the left half.

Find-Max(a, l, r):

```

1  let  $n \leftarrow |a|$ 
2  while  $l \leq r$ :
3      let  $\text{mid} \leftarrow \lfloor \frac{l+r}{2} \rfloor$ 
4      let  $\text{prev} \leftarrow (\text{mid} - 1 + n) \bmod n$ 
5      let  $\text{next} \leftarrow (\text{mid} + 1) \bmod n$ 
6
7      if  $a[\text{mid}] > a[\text{prev}]$  and  $a[\text{mid}] > a[\text{next}]$ :
8          return  $a[\text{mid}]$ 
9      else if  $a[\text{mid}] < a[\text{next}]$ :
10          $l \leftarrow \text{mid} + 1$ 
11     else:
12          $r \leftarrow \text{mid} - 1$ 
```

Each iteration discards half of the search range, so the running time is $\Theta(\log n)$.

Remark**思路引導**

Binary search 能成立，依賴一個關鍵不變量：比較 $a[\text{mid}]$ 與相鄰元素，可以確定最大值在哪一側。

(1) 若 $a[\text{mid}] > a[\text{prev}]$ 且 $a[\text{mid}] > a[\text{next}]$ ： $a[\text{mid}]$ 同時比左右鄰居大，符合局部最大值定義，即為全局最大值 (unimodal 只有一個峰)。

(2) 若 $a[\text{mid}] < a[\text{next}]$ ：目前在上坡段，峰在右側，搜尋範圍縮至 $[\text{mid} + 1, r]$ 。

(3) 否則 ($a[\text{mid}] > a[\text{next}]$, 且未滿足第一條) : 目前在下坡段, 峰在左側, 搜尋範圍縮至 $[l, \text{mid} - 1]$ 。

每次迭代搜尋範圍減半, 故時間複雜度為 $\Theta(\log n)$ 。

注意 prev/next 使用 $\text{mod } n$ 處理環狀邊界, 但 l, r 仍是線性索引——這是因為 unimodal 的定義保證: 不論從哪個位置開始, 上述判斷都能正確指出峰的方向。

7. (10 pts)

Show that to find the second largest one of a list of n numbers, we need at least $n - 2 + \lceil \log n \rceil$ comparisons.

Reference Answer

Claim. Any comparison-based algorithm finding the second largest of n distinct elements requires at least $n - 2 + \lceil \log n \rceil$ comparisons in the worst case.

Proof. Let w be the maximum, $S(w)$ the set of elements directly defeated by w , and $t = |S(w)|$. Partition all comparisons into those involving w and those not involving w .

Comparisons involving w : at least t and $t \geq \lceil \log n \rceil$. Each such comparison adds one element to $S(w)$, so the count is exactly t . To bound t : assign each element weight 1 initially; when w beats an element, merge weights. The adversary always makes the heavier element win, so w 's weight at most doubles per comparison. Since w must accumulate weight n ,

$$n \leq 2^t \implies t \geq \lceil \log n \rceil.$$

Comparisons not involving w : at least $n - 2$. The second largest must lie in $S(w)$: any element never directly beaten by w either lost to some non- w element (hence provably not second-largest) or was never shown inferior to w at all. Therefore the algorithm must identify $\max S(w)$. Two disjoint demands are placed on comparisons not involving w :

- **Identify $\max S(w)$:** finding the best among t candidates requires $\geq t - 1$ comparisons, all within $S(w)$.
- **Eliminate the $n - 1 - t$ remaining elements** (those beaten by non- w elements): each must lose at least once in a comparison involving at least one non- $S(w)$ element, giving $\geq n - 1 - t$ such comparisons.

These two sets of comparisons are disjoint—the first involves only $S(w)$ elements, the second involves at least one element outside $S(w)$ —so

$$\text{comparisons not involving } w \geq (t - 1) + (n - 1 - t) = n - 2.$$

Total: $t + (n - 2) \geq \lceil \log n \rceil + n - 2 = n - 2 + \lceil \log n \rceil$. ■

Remark

思路引導

整體框架: 以「是否涉及 w 」為主要分割, 對兩側各給下界再相加。

(1) 涉及 w 的比較 (恰好 $t = |S(w)|$ 次): 用 adversary argument 得 $t \geq \lceil \log n \rceil$ 。對手策略: 每次讓「積分較高」者獲勝並合併積分, 積分代表曾擊敗 (含間接) 的元素數。 w 最終積分須達 n , 每勝至多翻倍, 故 $t \geq \lceil \log n \rceil$ 。

(2) 不涉及 w 的比較 (至少 $n - 2$ 次): 這批比較同時肩負兩個獨立任務:

- (i) 在 $S(w)$ 內找出最大值： t 個候選需要 $t - 1$ 次；
- (ii) 淘汰 $n - 1 - t$ 個「其他」元素：各需至少輸一次。

兩個任務涉及的元素集合不重疊 (前者純 $S(w)$ ，後者含 non- $S(w)$ 元素)，故下界直接相加得 $n - 2$ 。

$$\text{合計} \geq \lceil \log n \rceil + (n - 2) = n - 2 + \lceil \log n \rceil。$$